



Проектирование и разработка распределённого хранилища данных для семантической сети

Выполнил: Барсуков М.А. Р3415

Научный руководитель: Цопа Е.А.

Университет ИТМО, 2026 г.

Актуальность

Графы знаний — миллиарды рёбер

Wikidata, DBpedia, корпоративные базы знаний

Один узел не справляется

миллиарды рёбер не помещаются в одну машину

Хэш-шардирование рвёт связи

обходы — переходы между шардами, задержка растёт с глубиной

Открытые решения — компромисс

не дают одновременно распределенность, графовую модель и сериализуемость

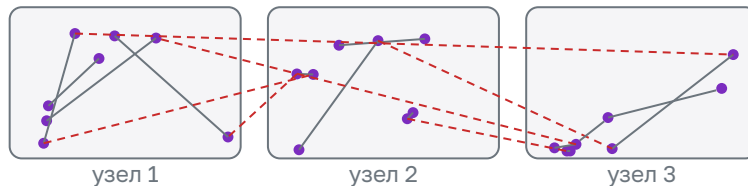
1 узел: переполнение



миллиарды рёбер → не помещается

наивное шардирование ↓

3 узла: связи разорваны



переходы между шардами → медленные обходы

Решение: семантически-осознанное размещение шардов

Цель: Обеспечение возможности горизонтального масштабирования хранения данных семантической сети.

Задачи:

1. Проанализировать существующие подходы к распределенному хранению RDF-данных, выявить их архитектурные ограничения.
2. Сформулировать требования к хранилищу и обосновать архитектуру.
3. Спроектировать архитектуру кластера.
4. Реализовать прототип с модулями хранения, кластеризацией и сетевым API.
5. Обеспечить покрытие ключевых модулей тестами (не менее 70%).
6. Нагрузочное тестирование и тестирование отказоустойчивости.
7. Сравнить полученные целевые характеристики с существующими подходами, оценить характеристики системы.

Анализ существующих решений

Система	Модель	Распределенность	Сериализуемость	Внешняя согласованность	Графовые операции
Virtuoso	RDF	●	●	○	●
Stardog	RDF	●	●	○	●
Blazegraph	RDF	●	●	○	●
Neo4j Enterprise	LPG	●	●	○	●
JanusGraph	LPG	●	○	○	●
Dgraph	LPG	●	●	○	●
Spanner	реляционная	●	●	●	○
CockroachDB	реляционная	●	●	●	○
rhizome	LPG	●	●	●	●

Незаполненная ниша: распределенное графовое хранилище с сериализуемостью и внешней согласованностью

Требования к системе

Функциональность

- **Графовая модель**
LPG, типизированные рёбра
- **MVCC и retention**
история 7 суток
- **Совместные запросы**
по графу и эмбедингам
- **API**
GraphQL

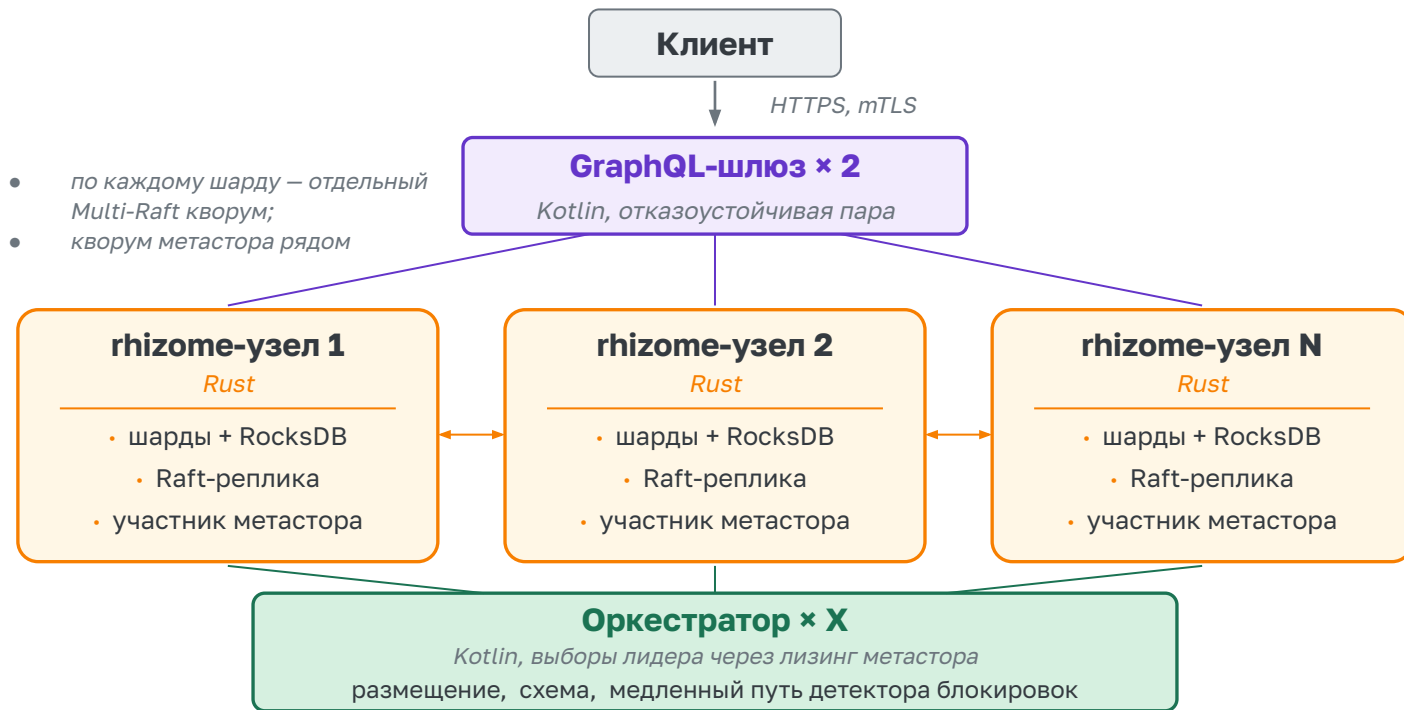
Гарантии корректности

- **Изоляция**
READ_COMMITTED, SNAPSHOT, SERIALIZABLE
- **Внешняя согласованность**
аналогичная подходу Spanner
- **Атомарность**
межшардовые транзакции
- **Долговечность**
Репликация на 3/5/N узлов

Эксплуатация

- **Масштабирование**
горизонтально на N узлов
- **Восстановление**
< 5 с после отказа
- **Безопасность**
взаимный TLS
- **Развёртывание**
commodity-серверы, без облачных API

Подзадача	Выбранное решение	Почему именно это
Модель хранения	LPG поверх RocksDB: граф-ориентированные семейства колонок, кэш ARC + предвыборка соседей, префиксные фильтры Блума, HNSW + PQ для векторов	Любой триплет RDF — частный случай LPG; графовые операции на уровне хранения, не поверх KV
Размещение	Алгоритм Leiden + планировщик NSGA-II по нагрузке	Развитие Louvain: устойчивые сообщества, корректные несвязные вершины; NSGA-II балансирует стоимость миграции
Репликация	raft-rs (TiKV/PingCAP) с отдельной группой на каждый шард, пакетная обработка с конвейеризацией и пре-выбор	Зрелая реализация (TiDB, etcd); joint consensus — без потерь доступности
Консенсус	Multi-Raft + встроенное хранилище метаданных (3 участника)	Нет узкого места общего консенсуса; хранилище метаданных встроено в data plane



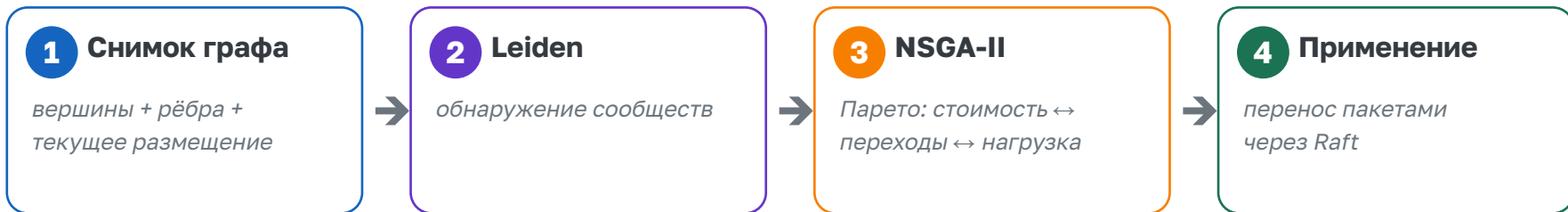
Семантическое размещение данных

Что делаем

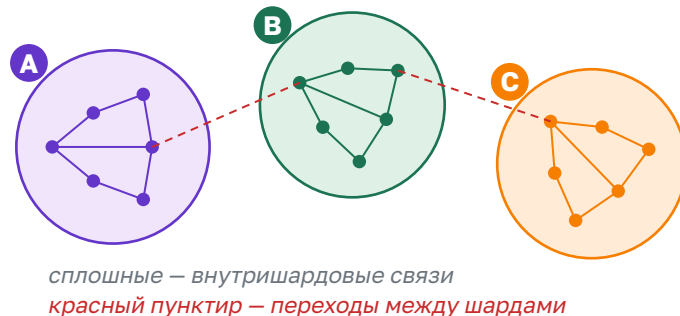
- **Leiden** — устойчивые сообщества, модулярность (вместо хэша)
- **NSGA-II** — Парето: стоимость $\leftrightarrow$ переходы $\leftrightarrow$ нагрузка
- **Порог миграции** — перенос — только при выгоде $> \Delta$

Стратегия	Доля межшардовых
Хэширование	$\approx 67\%$
По предикату	зависит от перекося
Leiden	$\approx 20\%$

Как работает планировщик

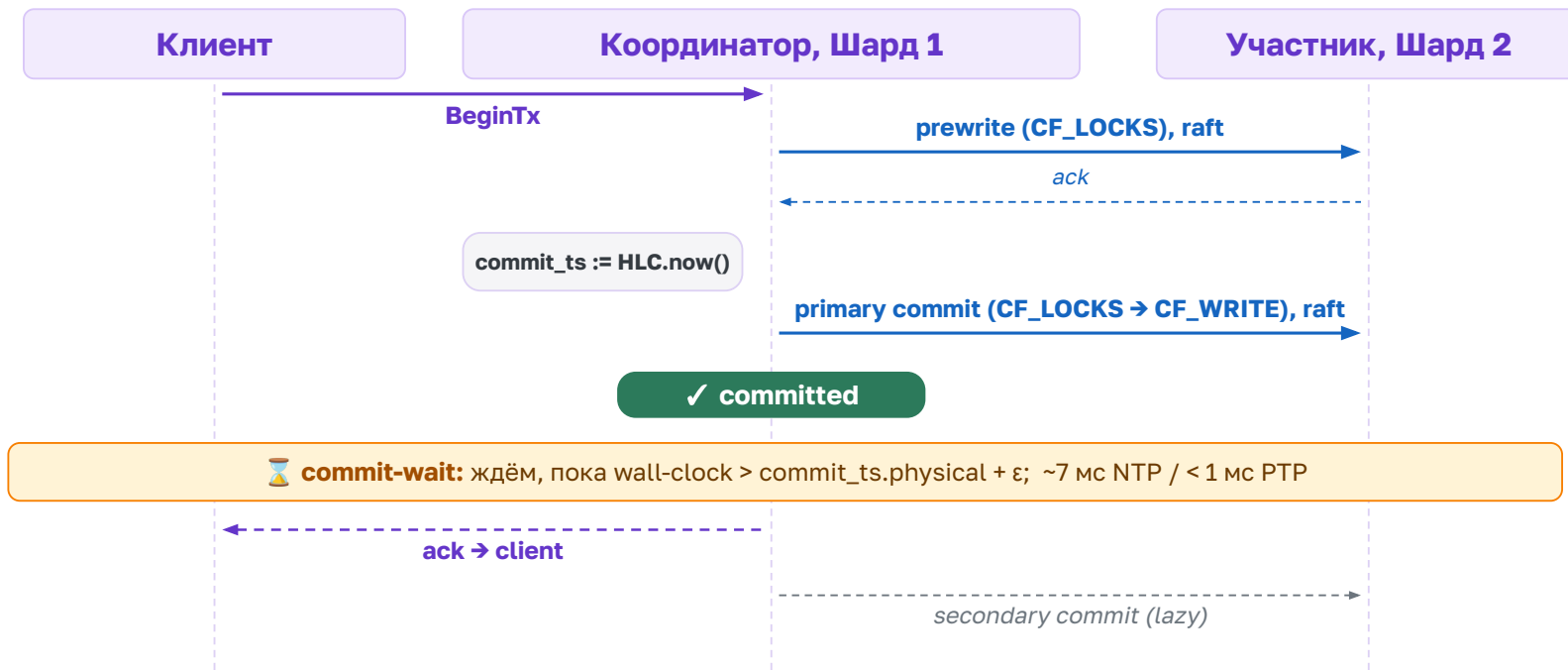


Сообщества Leiden — связанные концепты на одном шарде



Результат на BFS-бенчмарке: задержка обхода **-46...-52 %** против наивного хэш-шардинга

Семантика транзакций



SSI-проверка на prewrite (при конфликте — abort до точки committed): Cahill 2008, Peng–Dabek 2010, Spanner 2012

Гибридный детектор глобальных взаимных блокировок

Проблема: локальный детектор не видит цикл, проходящий через несколько шардов ($A \rightarrow B \rightarrow C \rightarrow A$) — это «замораживает» часть системы

БЫСТРЫЙ ПУТЬ

Зонд по цепочке ожидания

алгоритм Чанди–Мизра–Хааса

Узел замечает межшардовое ожидание → пускает зонд по цепочке. Зонд несёт стоимость каждого шага. Вернулся к инициатору → весь цикл известен → выбор жертвы.

Межшардовые циклы

≈ 5-50 мс

МЕДЛЕННЫЙ ПУТЬ

Обход глобального графа ожиданий

алгоритм Тарьяна — поиск сильно связанных компонент

Каждые 200 мс узлы отдают ожидания оркестратору. Тот склеивает граф и ищет циклы алгоритмом Тарьяна. Подхватывает то, что упустил быстрый путь.

Источник истины

< 1 с

Оба пути выбирают жертву одной формулой ($op_count \times age$) → двойной откат безопасен

Стоимостный выбор жертвы

минимальный ущерб — откатываем самую молодую или дешёвую

Постепенная деградация

оба пути независимы: паралич — только при одновременном отказе обоих

Идемпотентный откат

без распределённого CAS — двойное применение безопасно

Локальные циклы внутри одного шарда — отлавливаются встроенным графом ожиданий узла.

Качество реализации

4 сценария отказов

27 сквозных · pytest · docker-compose

746 модульных · Rust + Kotlin

покрытие 78 %

Отказ узла данных

< 2 с

перевыбор лидера Raft

Отказ оркестратора

< 5 с

лизинг подхватывает запасной

Отказ члена метастора

0 с простоя

кворум сохранён

Сетевой раскол

< 2 с

сторона с кворумом работает

60K

строк кода

Rust · Kotlin · Python

12

модулей

9 Rust + 3 Kotlin

11

RocksDB CF

MVCC · locks · WAL

Гарантии и инварианты

- **Типобезопасный инвариант**

без обработчика блокировок чтение не скомпилируется
→ устранение «осиротевших» блокировок — гарантия компилятора

- **Проверка SSI на предзаписи** — конфликт ловится до фиксации

- **Ожидание фиксации над HLC** — внешняя согласованность без Google TrueTime

- **Идемпотентный откат** — без распределённого CAS

Инструменты

- **Rust:** clippy · tarpaulin · criterion
- **Kotlin:** detekt · Kover · JUnit 5
- **Сквозные:** pytest + docker-compose
- **Хаос-тесты:** инъекция отказов
- **Сборка:** линтер, тесты, бенчмарки (+ cargo bench)

1. Алгоритмическая — гибридный детектор глобальных взаимных блокировок

Зонд Чанди–Мизра–Хааса *быстрый путь*

≈ 5–50 мс

- ✓ короткие циклы
- ✓ напрямую между узлами

Алгоритм Тарьяна *медленный путь*

≈ 400 мс – 1 с

- ✓ длинные циклы
- ✓ реплицированный граф ожиданий

Селектор жертвы · $op_count \times age$ · общий для control- и data-plane

идемпотентный откат *без распределённого CAS*

2. Архитектурная — сочетание свойств

✓ **Распределённость**

Multi-Raft

✓ **Графовая модель**

LPG

✓ **Serializable**

SSI

✓ **Внешняя согл.**

HLC + CW

Сочетание этих свойств не покрыто рассмотренными существующими открытыми решениями (**см. слайд 4**).

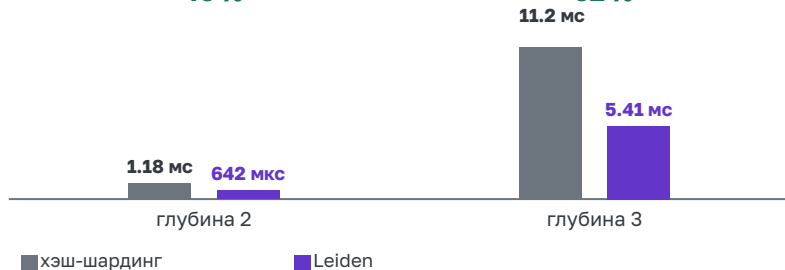
Результаты экспериментов

Задержка обхода графа (BFS)

хэш-шардинг vs Leiden · степень вершины 16 · 3 шарда

-46 %

-52 %

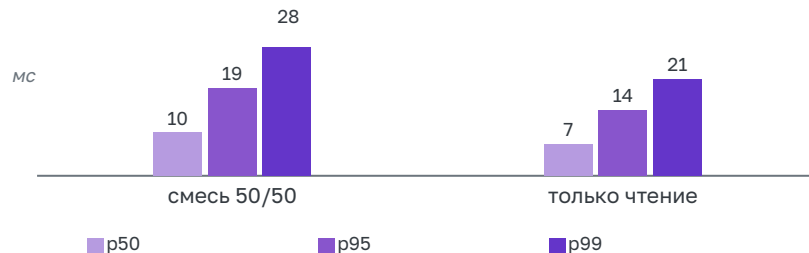


Задержка YCSB · перцентили p50 / p95 / p99

две нагрузки: смешанная 50 / 50 и только чтение

752 запр./с

1010 запр./с



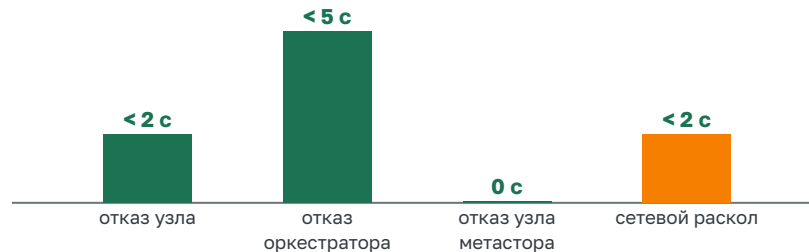
Массовая загрузка (импорт SST)

пропускная способность, млн записей/с



Отказоустойчивость · 4 сценария отказов

время восстановления, секунды



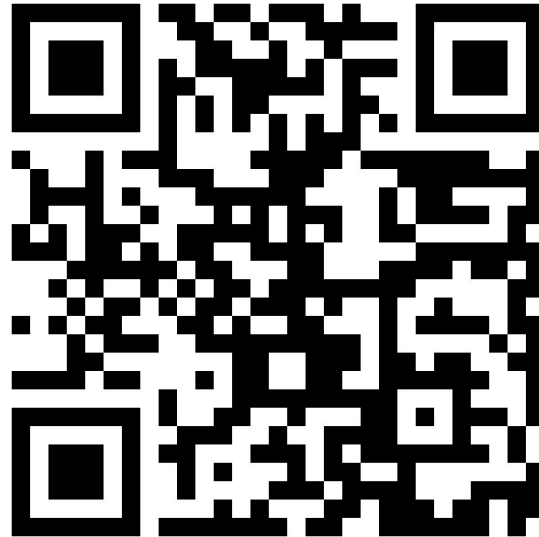
- Проанализированы существующие распределённые хранилища RDF- и LPG-данных; выявлены архитектурные ограничения и непокрытая ниша.
- Сформулированы требования к распределённому графовому хранилищу и обоснован выбор архитектурных решений.
- Спроектирована трёхуровневая архитектура кластера: GraphQL-шлюз, узлы хранения и оркестратор.
- Реализован прототип, на котором экспериментально подтверждена жизнеспособность предложенной архитектуры.
- Проведено нагрузочное и отказоустойчивое тестирование; получено снижение задержки обхода графа на 46–52 % относительно хэш-шардинга.

✓ 1. Анализ подходов	сл. 4
✓ 2. Требования	сл. 5
✓ 3. Архитектура	сл. 6, 7
✓ 4. Реализация прототипа	сл. 11
✓ 5. Покрытие тестами	сл. 11, 13
✓ 6. Тестирование под нагрузкой и отказы	сл. 13
✓ 7. Сравнение с существующими подходами	сл. 10, 13

Цель достигнута: возможность горизонтального масштабирования хранения данных семантической сети обеспечена, прототип развернут и протестирован.

**Спасибо
за внимание!**

it'sMO *re than a*
UNIVERSITY



github.com/maxbarsukov/rhizome